Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»
КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

К КУРСОВОЙ РАБОТЕ

НА ТЕМУ:

«Визуализация трёхмерного огня методом систем частиц с учётом параметров внешнего освещения»

Студент **ИУ7-53Б** (Группа) _____ (Подпись, дата) **А. К. Пресняков** (И. О. Фамилия)

Руководитель _____ (Подпись, дата) **А. В. Романов** (И. О. Фамилия)

2025 г.

РЕФЕРАТ

Расчетно-пояснительная записка 41 с., 18 рис., 4 табл., 9 источн., 1 прил.

СИСТЕМЫ ЧАСТИЦ, ВИЗУАЛИЗАЦИЯ ОГНЯ, ВНЕШНЕЕ ОСВЕЩЕНИЕ, МОДЕЛЬ ОСВЕЩЕНИЯ, Z-БУФЕР, ПРОИЗВОДИТЕЛЬНОСТЬ.

Цель работы – разработка программного обеспечения для визуализации трёхмерного огня методом систем частиц с учётом параметров внешнего освещения и исследование влияния количества частиц и параметров системы на производительность.

Для моделирования и отображения пламени использовался метод систем частиц, эмиссионная модель освещения с учётом внешнего точечного источника света и алгоритм z-буфера с аддитивным смешиванием для решения задачи видимости.

Разработаны формальные модели данных и схемы ключевых алгоритмов: основного цикла, обновления системы частиц, вычисления цвета частицы с учётом освещения, визуализации сцены. Реализовано спроектированное программное обеспечение на C++ с использованием фреймворка Qt, обеспечивающее интерактивное управление параметрами систем частиц, освещения и камеры в реальном времени. Описана структура сцены и форматы данных.

Проведено исследование производительности: проанализирована зависимость частоты кадров от общего количества частиц в сцене (от 10 000 до 2 000 000) и оценено влияние на производительность использования внешнего освещения и алгоритма z-буфера.

СОДЕРЖАНИЕ

РЕФЕРАТ **3**

ВВЕДЕНИЕ **6**

1 Аналитическая часть **7**

1.1 Постановка задачи визуализации и структура сцены 7

1.2 Метод систем частиц для моделирования пламени 7

1.2.1 Параметры частицы и динамика во времени 7

1.2.2 Параметры системы частиц 8

1.3 Параметры перспективной камеры 9

1.4 Альтернативные подходы к визуализации огня 9

1.4.1 Текстурные и спрайтовые анимации 9

1.4.2 Объёмные методы и объёмный рендеринг 10

1.4.3 Физически обоснованное моделирование на основе уравнений гидродинамики 10

1.5 Модели освещения 10

1.5.1 Классические локальные модели освещения 11

1.5.2 Освещение самосветящейся полупрозрачной среды 11

1.6 Алгоритмы удаления невидимых поверхностей 12

1.6.1 Постановка задачи видимости 12

1.6.2 Алгоритм Робертса 12

1.6.3 Алгоритм Варнока 13

1.6.4 Алгоритм z-буфера 13

1.7 Сравнительный анализ и выбор методов 14

1.8 Вывод 15

2 Конструкторская часть **16**

2.1 Требования к программному обеспечению 16

2.2 Декомпозиция программного обеспечения 16

2.3 Модель данных и структура сцены 17

2.3.1 Векторные и матричные величины 17

2.3.2 Частица 17

2.3.3 Система частиц 18

2.3.4 Источник света 18

2.3.5 Камера . . . 18
2.3.6 Сцена . . . 19
2.4 Основной цикл работы программы . . . 19
2.5 Алгоритм обновления системы частиц . . . 21
2.6 Алгоритм вычисления цвета частиц с учётом освещения . . . 22
2.7 Алгоритм визуализации сцены . . . 23
2.8 Вывод . . . 24

3 Технологическая часть . . . 25
3.1 Выбор языка программирования и средств разработки . . . 25
3.2 Организация модулей программы . . . 26
3.3 Структура классов программного обеспечения . . . 27
3.4 Реализация ключевых алгоритмов . . . 28
3.4.1 Алгоритм обновления системы частиц . . . 28
3.4.2 Алгоритм вычисления цвета частицы с учётом освещения . . . 29
3.4.3 Алгоритм z-буфера с аддитивным смешиванием . . . 30
3.5 Интерфейс пользователя . . . 31
3.6 Результаты работы программного обеспечения . . . 34
3.7 Вывод . . . 36

4 Исследовательская часть . . . 37
4.1 Методика исследования производительности . . . 37
4.2 Анализ производительности системы . . . 37
4.2.1 Зависимость частоты кадров от количества частиц . . . 37
4.2.2 Зависимость от наличия z-буфера . . . 38
4.3 Вывод . . . 39

ЗАКЛЮЧЕНИЕ . . . 40

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ . . . 41

ПРИЛОЖЕНИЕ А . . . 42

ВВЕДЕНИЕ

В последние десятилетия методы компьютерной графики получили широкое распространение в самых разных областях: от индустрии развлечений и компьютерных игр до инженерных расчётов, имитационного моделирования и систем визуализации научных данных. Существенную роль в формировании визуального восприятия сцены играют реалистичные специальные эффекты – такие, как дым, огонь, туман, взрывы и другие объекты сложной формы. Корректная визуализация подобных явлений позволяет повысить наглядность и выразительность трёхмерных сцен, однако их моделирование связано с рядом вычислительных и алгоритмических сложностей.

Одним из наиболее распространённых подходов к отображению огня и других подобъектов является метод систем частиц. В рамках этого подхода сложный объект представляется большим количеством элементарных частиц, каждая из которых обладает собственной позицией, скоростью, временем жизни и набором визуальных параметров. Совокупное поведение частиц позволяет воспроизвести характерную динамику и структуру пламени.

В рамках данной курсовой работы рассматривается задача визуализации трёхмерного огня методом систем частиц с учётом параметров внешнего освещения. Объектом исследования в работе является программная система визуализации трёхмерной сцены с анимированными эффектами огня. Предметом исследования являются методы визуализации огня на основе систем частиц и упрощённые модели внешнего освещения, применяемые при рендеринге частиц.

Целью курсовой работы является разработка программного обеспечения для визуализации трёхмерного огня методом систем частиц с учётом внешнего освещения и в исследовании влияния количества частиц и параметров системы на производительность.

Для достижения поставленной цели необходимо выполнить следующие задачи:

1) провести сравнение существующих методов визуализации систем частиц и моделей освещения, применяемых для отображения огня и схожих по природе явлений, с точки зрения визуального качества и вычислительной сложности. Выбрать и обосновать алгоритмы визуализации частиц и модель освещения, обеспечивающие требуемый баланс между производительностью и реалистичностью изображения.

2) разработать структуру программного обеспечения, обеспечивающие представление частиц, систем частиц, камеры, источников света и сцены в целом.

3) реализовать программный комплекс, позволяющий пользователю создавать и настраивать несколько независимых систем частиц (скорость, размер, цвет, количество частиц), а также управлять параметрами внешнего освещения и виртуальной камеры в реальном времени.

4) провести исследование зависимости частоты кадров от общего количества частиц в сцене и выбранных алгоритмов визуализации, проанализировать полученные результаты и сформулировать выводы о влиянии выбранных алгоритмов и настроек на производительность.

1 Аналитическая часть

В аналитической разделе будут рассматриваются основные подходы к визуализации огня и сформулированы решения, принимаемые при разработке программного обеспечения

1.1 Постановка задачи визуализации и структура сцены

Рассматривается трёхмерная сцена, в которой необходимо визуализировать один или несколько источников пламени. Каждый источник огня моделируется как независимая система частиц, эволюционирующих во времени под действием заданных правил. Помимо систем частиц, сцена включает внешний источник света и виртуальную камеру, определяющую точку зрения наблюдателя.

Состояние сцены на дискретном шаге времени можно описать следующими компонентами:

1) множеством систем частиц, каждая из которых задаёт отдельный источник пламени;

2) одним точечным источником внешнего света с настраиваемыми положением и интенсивностью;

3) виртуальной камерой с настраиваемыми положением и параметрами проекции;

4) прямоугольной областью вывода, соответствующей окну приложения.

Задача визуализации состоит в том, чтобы по текущему состоянию систем частиц, источника света и камеры формировать последовательность двумерных кадров, создающих впечатление наблюдения за трёхмерным огнём. При этом должны обеспечиваться:

1) визуально правдоподобное поведение пламени;

2) возможность независимой настройки параметров каждой системы частиц (количество частиц, скорость, размер, цвет, характеристики эмиссии);

3) управляемость параметров внешнего освещения и виртуальной камеры в реальном времени;

1.2 Метод систем частиц для моделирования пламени

Метод систем частиц основан на представлении сложного по форме и динамике объекта набором элементарных частиц, каждая из которых имеет заданное состояние и подчиняется правилам эволюции. [1] Для моделирования огня такой подход позволяет воспроизводить форму факела, мерцание и затухание пламени без физически точного моделирования процессов горения.

1.2.1 Параметры частицы и динамика во времени

Состояние одной частицы удобно описывать вектором

$$x(t) = \big(\mathbf{p}(t), \mathbf{v}(t), \tau(t), s(t), \alpha(t), \mathbf{C}(t)\big), \tag{1.1}$$

где:

1) $\mathbf{p}(t) = (x(t), y(t), z(t))$ – положение частицы в мировых координатах;
2) $\mathbf{v}(t)$ – вектор скорости;
3) $\tau(t) \in [0, 1]$ – нормированная доля оставшейся жизни частицы;
4) $s(t)$ – линейный размер частицы;
5) $\alpha(t) \in [0, 1]$ – коэффициент прозрачности;
6) $\mathbf{C}(t)$ – цветовой вектор (например, в пространстве RGB).

На каждом шаге моделирования выполняется дискретное обновление состояния по времени с шагом Δt:

$$\mathbf{p}(t + \Delta t) = \mathbf{p}(t) + \mathbf{v}(t)\,\Delta t, \tag{1.2}$$

$$\mathbf{v}(t + \Delta t) = \mathbf{v}(t) + \mathbf{a}(t)\,\Delta t, \tag{1.3}$$

где $\mathbf{a}(t)$ – результирующее ускорение, включающее, например, подъёмную силу, действующую вверх, и псевдослучайные возмущения, имитирующие турбулентность [2].

Нормированная жизнь частицы уменьшается по закону:

$$\tau(t + \Delta t) = \max\bigl(0,\ \tau(t) - k_\tau\,\Delta t\bigr), \tag{1.4}$$

где $k_\tau > 0$ – параметр, определяющий среднюю продолжительность существования частицы. При достижении $\tau = 0$ частица считается мёртвой и подлежит удалению из системы.

Визуальные параметры задаются как функции от τ:

$$s(t) = s_0\, f_s\bigl(\tau(t)\bigr), \quad \alpha(t) = f_\alpha\bigl(\tau(t)\bigr), \quad \mathbf{C}(t) = \mathcal{C}\bigl(\tau(t)\bigr), \tag{1.5}$$

где s_0 – базовый размер частицы, а функции f_s, f_α и $\mathcal{C}$ задают законы изменения размера, прозрачности и цвета в зависимости от стадии “жизни” частицы. [6]

Для пламени функция $\mathcal{C}(\tau)$ обычно определяет переход от более горячих оттенков (белый, жёлтый) при больших значениях τ к более холодным (оранжевый, красный, дымный) при малых значениях τ.

1.2.2 Параметры системы частиц

Система частиц задаёт множество частиц и параметры их рождения в сцене. В общем виде систему можно описать набором:

$$P = \bigl(\Omega, \lambda, N_{\max}, \Theta\bigr), \tag{1.6}$$

где:

1) $\Omega \subset \mathbb{R}^3$ – область эмиссии частиц (эмиттер), например круг или диск на плоскости;
2) λ – средняя интенсивность эмиссии (число рождающихся частиц в единицу времени);

3) $N_{\max}$ – максимальное число частиц в системе;

4) Θ – набор параметров, определяющих начальные распределения скоростей, времени жизни и базовых цветов частиц.

Процесс эволюции системы частиц на каждом шаге включает:

1) эмиссию новых частиц в области Ω до достижения $N_{\max}$;

2) обновление состояния всех живых частиц по формулам (2.2)–(2.4);

3) пересчёт визуальных параметров $s(t)$, $\alpha(t)$ и $\mathbf{C}(t)$ по формулам (2.5);

4) удаление частиц с $\tau = 0$.

Такая схема позволяет описывать различные типы источников пламени, изменяя только параметры эмиттера и законы изменения визуальных характеристик.

1.3 Параметры перспективной камеры

Математически перспективная проекция задаётся усечённой пирамидой видимости (view frustum), которая определяется набором параметров [2]:

— $\mathbf{p}_C = (x_c, y_c, z_c)$ – положение камеры в мировой системе координат.

— $\mathbf{d}_C$ – нормализованный вектор направления взгляда.

— $\mathbf{u}_C$ – вектор «вверх», задающий ориентацию камеры вокруг оси взгляда.

— φ – угол поля зрения по вертикали . Определяет «широкоугольность» камеры. Большие значения φ дают эффект панорамы, малые — эффект телеобъектива.

— $a = w/h$ – соотношение сторон целевого изображения.

— n, f – расстояния до ближней и дальней плоскостей отсечения. Объекты, находящиеся ближе n или дальше f, не отображаются.

На основе этих параметров формируются матрицы преобразования: матрица вида M_{view} (переход из мировых координат в координаты камеры) и матрица перспективной проекции M_{proj}.

1.4 Альтернативные подходы к визуализации огня

Системы частиц являются лишь одним из распространённых подходов к визуализации огня. Для полноты картины кратко рассмотрены альтернативы и обоснован выбор именно данного метода.

1.4.1 Текстурные и спрайтовые анимации

Текстурные и спрайтовые анимации основаны на использовании заранее подготовленных последовательностей изображений огня, отображаемых на плоских полигонах или билбордах [4]. Движение пламени формируется за счёт смены кадров анимации.

Преимущества:

1) крайне простая реализация;

2) минимальная вычислительная сложность;

3) предсказуемое визуальное качество.

Недостатки:

1) привязка к заранее подготовленным анимациям;

2) ограниченные возможности настройки формы и поведения пламени;

3) слабая интеграция с динамическим освещением.

1.4.2 Объёмные методы и объёмный рендеринг

В объёмных методах огонь представляется в виде трёхмерного поля плотности, температуры или коэффициентов эмиссии и поглощения, заданного на регулярной воксельной сетке [5]. Визуализация выполняется методом объёмного рендеринга, предполагающим интегрирование вклада среды вдоль луча зрения.

Преимущества:

1) высокая степень реализма, возможность учёта самозатенения и рассеяния;

2) естественное представление сложной структуры пламени.

Недостатки:

1) необходимость хранения и обновления трёхмерных полей;

2) значительная вычислительная сложность процедуры рендеринга;

3) повышенные требования к аппаратным ресурсам.

1.4.3 Физически обоснованное моделирование на основе уравнений гидродинамики

Физически обоснованные модели описывают движение и нагрев газовой среды с помощью уравнений гидродинамики и уравнений переноса энергии, которые затем используются для визуализации пламени [4].

Преимущества:

1) физическая достоверность поведения огня;

2) возможность моделирования взаимодействия потока с препятствиями.

Недостатки:

1) сложность реализации и настройки численных методов;

2) высокие требования к вычислительным ресурсам;

3) неочевидная связь физических параметров с удобными для пользователя визуальными параметрами.

1.5 Модели освещения

Модель освещения определяет, как параметры источников света и визуальные свойства объектов сцены преобразуются в наблюдаемый цвет на экране. Для огня важно учесть, что он

является самосветящейся полупрозрачной средой, на которую дополнительно влияет внешний источник света.

1.5.1 Классические локальные модели освещения

В традиционных трёхмерных сценах широко используются локальные модели освещения, такие как диффузная модель Ламберта и модель Фонга. [4] В диффузной модели яркость точки поверхности определяется выражением

$$I_{\text{diff}} = I_L k_d \max\bigl(0, \mathbf{n} \cdot \mathbf{l}\bigr), \tag{1.7}$$

где I_L – интенсивность источника света, k_d – диффузный коэффициент материала, $\mathbf{n}$ – единичный вектор нормали к поверхности, $\mathbf{l}$ – единичный вектор, направленный на источник.

Модель Фонга добавляет зеркальную составляющую:

$$I_{\text{spec}} = I_L k_s \max\bigl(0, \mathbf{r} \cdot \mathbf{v}\bigr)^n, \tag{1.8}$$

где k_s – зеркальный коэффициент, $\mathbf{r}$ – направление отражённого луча, $\mathbf{v}$ – направление на наблюдателя, n – показатель блеска. [4]

Эти модели предполагают наличие определённой геометрической поверхности и нормали, что не вполне соответствует частицам как элементам объёмной полупрозрачной среды.

1.5.2 Освещение самосветящейся полупрозрачной среды

Огонь можно рассматривать как самосветящуюся среду, в которой каждая частица сама испускает свет, а внешний источник света лишь модифицирует наблюдаемую яркость [5]. В приближении цвет частицы определяется эмиссионной составляющей и коэффициентом, зависящим от внешнего освещения.

Пусть $\mathbf{C}_{\text{emiss}}$ – эмиссионный цвет частицы, определяемый её временем жизни и базовым цветом системы, а внешнее освещение учитывается через коэффициент затухания по расстоянию до собственного источника света:

$$a(d) = \frac{1}{1 + kd^2}, \tag{1.9}$$

где d — расстояние от частицы до источника, $k > 0$ – параметр затухания. [4]

Тогда итоговый цвет частицы можно задать как

$$\mathbf{C}_{\text{final}} = \mathbf{C}_{\text{emiss}} \odot \bigl(\mathbf{1} + \beta\, a(d)\bigr), \tag{1.10}$$

где $\mathbf{1} = (1, 1, 1)$, β – коэффициент влияния внешнего света, а символ $\odot$ обозначает поэлементное умножение компонент цвета. [6]

1.6 Алгоритмы удаления невидимых поверхностей

При визуализации трёхмерной сцены необходимо определить, какие элементы должны быть видимы в каждом пикселе изображения. Это приводит к задаче удаления невидимых линий и поверхностей [2]. В рассматриваемой сцене присутствуют как потенциально непрозрачные элементы (фон), так и большое количество полупрозрачных частиц, моделирующих пламя.

1.6.1 Постановка задачи видимости

После перехода к координатам камеры и применения проекции каждая вершина полигона или частицы имеет экранные координаты (x, y) и значение глубины z, характеризующее расстояние до наблюдателя [3]. Для каждого пикселя (x, y) требуется выбрать фрагмент с минимальным значением z среди всех фрагментов, проецирующихся в данный пиксель:

$$F^*(x, y) = \arg\min_{F_k} z_k(x, y). \tag{1.11}$$

Если грань объекта лежит на плоскости

$$Ax + By + Cz + D = 0, \tag{1.12}$$

то при фиксированных x и y глубина точки этой грани может быть получена как

$$z(x, y) = -\frac{Ax + By + D}{C}, \tag{1.13}$$

что используется в ряде алгоритмов определения видимости. [3]

1.6.2 Алгоритм Робертса

Алгоритм Робертса относится к числу ранних методов удаления невидимых линий и поверхностей и основан на попарном анализе взаимного перекрытия полигонов в проекции [3]. В общем виде алгоритм включает:

1) вычисление уравнений плоскостей для всех граней объектов сцены;

2) проецирование граней на плоскость экрана и определение областей пересечения их проекций;

3) сравнение значений глубины $z_i(x, y)$ и $z_j(x, y)$ для граней P_i и P_j в характерных точках областей пересечения;

4) при необходимости – разбиение граней на части, соответствующие видимым и невидимым областям.

Математически видимость в точке экрана определяется сравнением значений

$$z_i(x, y) = -\frac{A_i x + B_i y + D_i}{C_i}, \quad z_j(x, y) = -\frac{A_j x + B_j y + D_j}{C_j}. \tag{1.14}$$

Преимущество алгоритма Робертса – учёт взаимного перекрытия граней; недостатки связаны с вычислительной сложностью и трудоёмкостью реализации при большом количестве полигонов [3].

1.6.3 Алгоритм Варнока

Алгоритм Варнока относится к подрастровым методам и основан на рекурсивном разбиении области экрана на более мелкие прямоугольники. [4] Для исходного окна изображения R_0 определяется множество полигонов, проецирующихся в эту область. Далее выполняются следующие шаги:

1) если в R_0 не попадает ни один полигон, область заполняется фоновым цветом;

2) если в R_0 проецируется один полигон и полностью покрывает его, область заполняется цветом этого полигона;

3) в общем случае R_0 разбивается, как правило, на четыре подокна, и процедура рекурсивно повторяется для каждого подокна.

При этом анализируется пересечение полигонов с подокнами, а глубина в характерных точках (например, в центре подокна) вычисляется по уравнению вида [3].

Алгоритм Варнока обеспечивает адаптивное уточнение решения в областях с высокой геометрической сложностью, но требует значительного числа операций пересечения и обрезки полигонов, что затрудняет его применение в динамических сценах с большим числом примитивов.

1.6.4 Алгоритм z-буфера

Алгоритм z-буфера реализует решение задачи видимости непосредственно в растровом пространстве [3]. Для каждого пикселя (x, y) поддерживаются значения глубины ближайшего уже отрисованного фрагмента $z_{\text{buf}}(x, y)$ и его цвета $C_{\text{buf}}(x, y)$.

Начальная инициализация имеет вид

$$z_{\text{buf}}(x, y) = z_{\max}, \quad C_{\text{buf}}(x, y) = C_{\text{background}}, \tag{1.15}$$

где $z_{\max}$ – значение глубины на дальней плоскости отсечения.

При обработке нового фрагмента с глубиной z_{new} и цветом C_{new} выполняется сравнение:

$$\text{если } z_{\text{new}} < z_{\text{buf}}(x, y), \tag{1.16}$$

$$\text{то } z_{\text{buf}}(x, y) := z_{\text{new}}, \quad C_{\text{buf}}(x, y) := C_{\text{new}}, \tag{1.17}$$

иначе фрагмент считается невидимым и отбрасывается.

Преимущества z-буфера:

1) отсутствие необходимости глобальной сортировки объектов по глубине;

2) масштабируемость при большом числе простых примитивов. [3]

1.7 Сравнительный анализ и выбор методов

Для обоснованного выбора ключевых методов визуализации был проведён сравнительный анализ по сформулированным критериям, наиболее значимым для задач моделирования огня системами частиц в реальном времени.

В таблице 1.1 представлено сравнение рассмотренных моделей освещения.

Таблица 1.1 — Сравнительный анализ моделей освещения для систем частиц

Критерий / Модель	**Локальные**	**Объёмная**	**Эмиссионная**
Адекватность физике огня	–	+	+
Учёт самосвечения	–	+	+
Учёт внешнего освещения	+	+	+
Вычислительная эффективность	+	–	+

Классические локальные модели неприменимы к частицам из-за отсутствия у них поверхности и нормали. Объёмная модель физически наиболее адекватна, но обладает неприемлемо высокой вычислительной сложностью. Принятая эмиссионная модель обеспечивает баланс: она учитывает самосвечение и внешний свет, оставаясь при этом вычислительно эффективной, что соответствует требованиям интерактивного приложения.

В таблице 1.2 представлено сравнение рассмотренных алгоритмов удаления невидимых поверхностей.

Таблица 1.2 — Сравнительный анализ алгоритмов удаления невидимых поверхностей

Критерий / Алгоритм	**Робертса**	**Варнока**	**Z-буфер (принятый)**
Производительность при большом N	–	–	+
Поддержка полупрозрачности	–	–	+ (с аддитивным смешиванием)
Применимость к динамической сцене	–	–	+

Алгоритмы Робертса и Варнока демонстрируют низкую производительность при большом количестве примитивов и высокую сложность реализации, что делает их неподходящими для динамических систем частиц. Алгоритм z-буфера лишён этих недостатков: его производительность линейно зависит от числа примитивов, он адаптируется к динамическим сценам. В сочетании с аддитивным смешиванием цветов он обеспечивает корректное отображение полупрозрачных частиц пламени [5].

1.8 Вывод

В аналитической части были рассмотрены основные подходы к визуализации огня и сформулированы решения, принимаемые при разработке программного обеспечения.

2 Конструкторская часть

В конструкторском разделе рассматриваются требования к программному обеспечению для визуализации трёхмерного огня методом систем частиц, разработана общая архитектура и формальная модель данных, описывающая основные объекты сцены.

2.1 Требования к программному обеспечению

Целью проектируемого программного обеспечения является визуализация трёхмерного огня методом систем частиц с учётом параметров внешнего освещения и возможностью интерактивной настройки параметров сцены пользователем.

К проектируемому программному обеспечению предъявляются следующие функциональные требования:

1) предоставление пользователю возможности создавать и удалять несколько независимых систем частиц, каждая из которых моделирует отдельный источник пламени;
2) предоставление возможности изменения параметров каждой системы частиц: максимального количества частиц, начальной скорости, размера и базового цвета частиц;
3) реализация внешнего точечного источника света с настраиваемыми положением и интенсивностью;
4) предоставление средств управления положением и ориентацией виртуальной камеры (масштабирование, поворот);
5) отображение трёхмерной сцены с учётом выбранной модели освещения и алгоритма удаления невидимых поверхностей;
6) обновление изображения сцены в реальном времени при изменении параметров систем частиц, источника света и камеры.

Пользовательский интерфейс должен удовлетворять следующим требованиям:

1) наличие области визуализации сцены, занимающей основную часть окна;
2) наличие панели управления параметрами систем частиц (количество частиц, скорость, размер, цвет);
3) наличие панели управления параметрами внешнего источника света (координаты, интенсивность);
4) наличие панели управления параметрами виртуальной камеры (позиция, угол обзора, ориентация);
5) отображение текущих значений параметров и возможность их изменения через стандартные элементы управления (ползунки, поля ввода).

2.2 Декомпозиция программного обеспечения

Для обеспечения требуемой функциональности программное обеспечение делится на ряд логически обособленных подсистем.

1) модули моделирования систем частиц;
2) модули визуализации;
3) модули математической обработки необходимых структур данных;
4) модуль пользовательского интерфейса и обработки событий.

2.3 Модель данных и структура сцены

Для проектируемого программного обеспечения были описаны собсвтенные типы данных для корректной работы с математическими преобразованиями и методами визуализации.

2.3.1 Векторные и матричные величины

Для представления пространственных величин используется трёхмерный вектор

$$\mathbf{v} = (v_x, v_y, v_z), \tag{2.1}$$

где v_x, v_y, v_z – действительные компоненты.

Преобразования координат реализуются с помощью матриц 4×4 в однородных координатах:

$$\tilde{\mathbf{p}}' = M\tilde{\mathbf{p}}, \tag{2.2}$$

где $\tilde{\mathbf{p}} = (x, y, z, 1)^T$ — вектор однородных координат точки, M — матрица преобразования (мировая, видовая или проекционная) [3].

2.3.2 Частица

Объект "Частица"описывается кортежем

$$\text{Particle} = \big(\mathbf{p}, \mathbf{v}, \tau, s, \alpha, \mathbf{C}\big), \tag{2.3}$$

где:

1) $\mathbf{p}$ – положение частицы в мировых координатах;
2) $\mathbf{v}$ – скорость;
3) τ – нормированная оставшаяся жизнь;
4) s — размер;
5) α – прозрачность;
6) $\mathbf{C}$ – цветовой вектор.

Над объектом "Частица"определены операции обновления состояния (обновление положения, скорости и τ) и вычисления визуальных параметров.

2.3.3 Система частиц

Объект "Система частиц"задаётся кортежем

$$\text{PSystem} = \big(\Omega, \lambda, N_{\max}, \Theta, \mathcal{S}\big), \tag{2.4}$$

где:

1) Ω – область эмиссии частиц;
2) λ – интенсивность эмиссии;
3) $N_{\max}$ – максимальное число частиц;
4) Θ – набор параметров, определяющих начальные распределения скоростей, времени жизни и базового цвета;
5) $\mathcal{S}$ – множество текущих частиц, принадлежащих системе.

Над системой частиц определены операции эмиссии новых частиц, обновления всех частиц и удаления частиц с $\tau = 0$.

2.3.4 Источник света

Объект "Источник света"представляет собой точечный источник и описывается кортежем

$$\text{Light} = \big(\mathbf{p}_L, I_L, k\big), \tag{2.5}$$

где:

1) $\mathbf{p}_L$ — положение источника света в мировых координатах;
2) I_L — базовая интенсивность;
3) k — параметр затухания освещённости по расстоянию.

2.3.5 Камера

Объект "Камера"описывается положением, ориентацией и параметрами проекции:

$$\text{Camera} = \big(\mathbf{p}_C, \mathbf{d}_C, \mathbf{u}_C, \varphi, n, f\big), \tag{2.6}$$

где:

1) $\mathbf{p}_C$ — положение камеры;
2) $\mathbf{d}_C$ — направление взгляда;
3) $\mathbf{u}_C$ — вектор «вверх» камеры;
4) φ — угол поля зрения;
5) n, f — расстояния до ближней и дальней плоскостей отсечения.

На основе этих параметров формируются матрицы вида и проекции, используемые модулем визуализации.

2.3.6 Сцена

Объект "Сцена"объединяет все элементы:

$$\text{Scene} = \big(\{\text{PSystem}_i\}, \text{Light}, \text{Camera}\big), \tag{2.7}$$

где множество $\{\text{PSystem}_i\}$ содержит все системы частиц, присутствующие в текущий момент.

2.4 Основной цикл работы программы

Работа программного обеспечения организована в виде основного цикла, в рамках которого последовательно выполняются обработка событий пользовательского интерфейса, обновление состояния сцены и визуализация трёхмерного изображения.

Структурное представление основного процесса визуализации огня в нотации IDEF0 (диаграмма нулевого уровня) приведено на рисунке 2.1.



Рисунок 2.1 — Диаграмма IDEF0 нулевого уровня процесса визуализации трёхмерного огня

Полное представление основного процесса работы программы представлено в расширенной нотации IDEF0 (диаграмма первого уровня) приведено на рисунке 2.2.

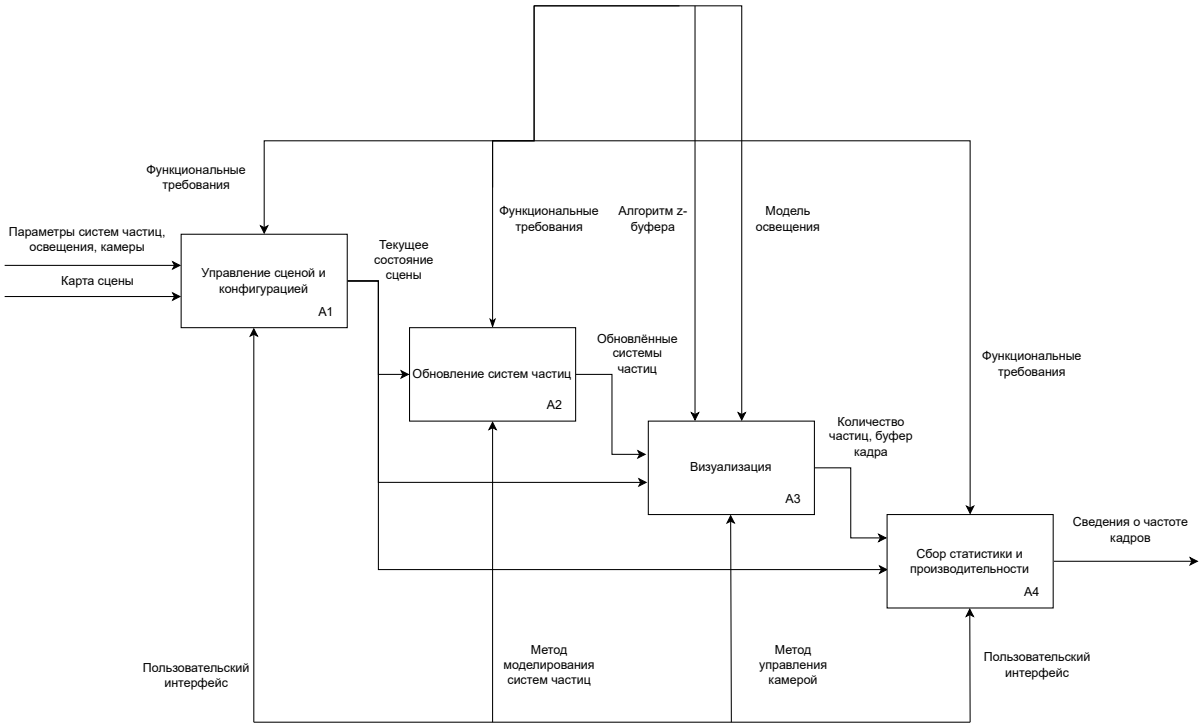


Рисунок 2.2 — Диаграмма IDEF0 первого уровня процесса работы программного обеспечения

2.5 Алгоритм обновления системы частиц

Алгоритм обновления одной системы частиц реализует эволюцию частиц во времени: рождение новых частиц, изменение их положения и характеристик, удаление мёртвых частиц.

Схема алгоритма обновления системы части приведена на рисунке 2.3.

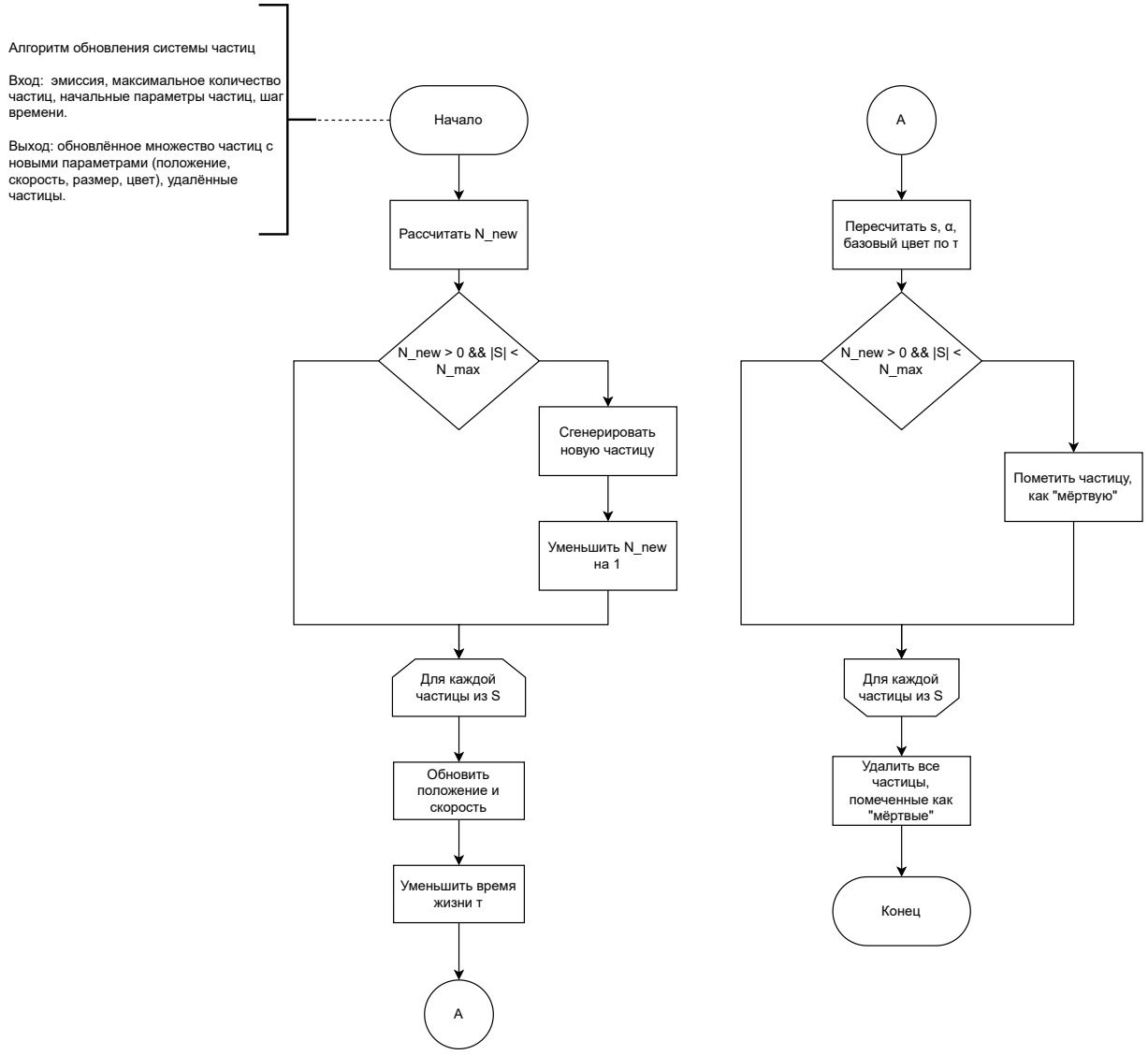


Рисунок 2.3 — Схема алгоритма обновления системы частиц

2.6 Алгоритм вычисления цвета частиц с учётом освещения

Алгоритм вычисления цвета частиц реализует принятую в работе эмиссионную модель освещения с учётом внешнего точечного источника света.

Схема алгоритма вычисления итогового цвета частицы приведена на рисунке 2.4.

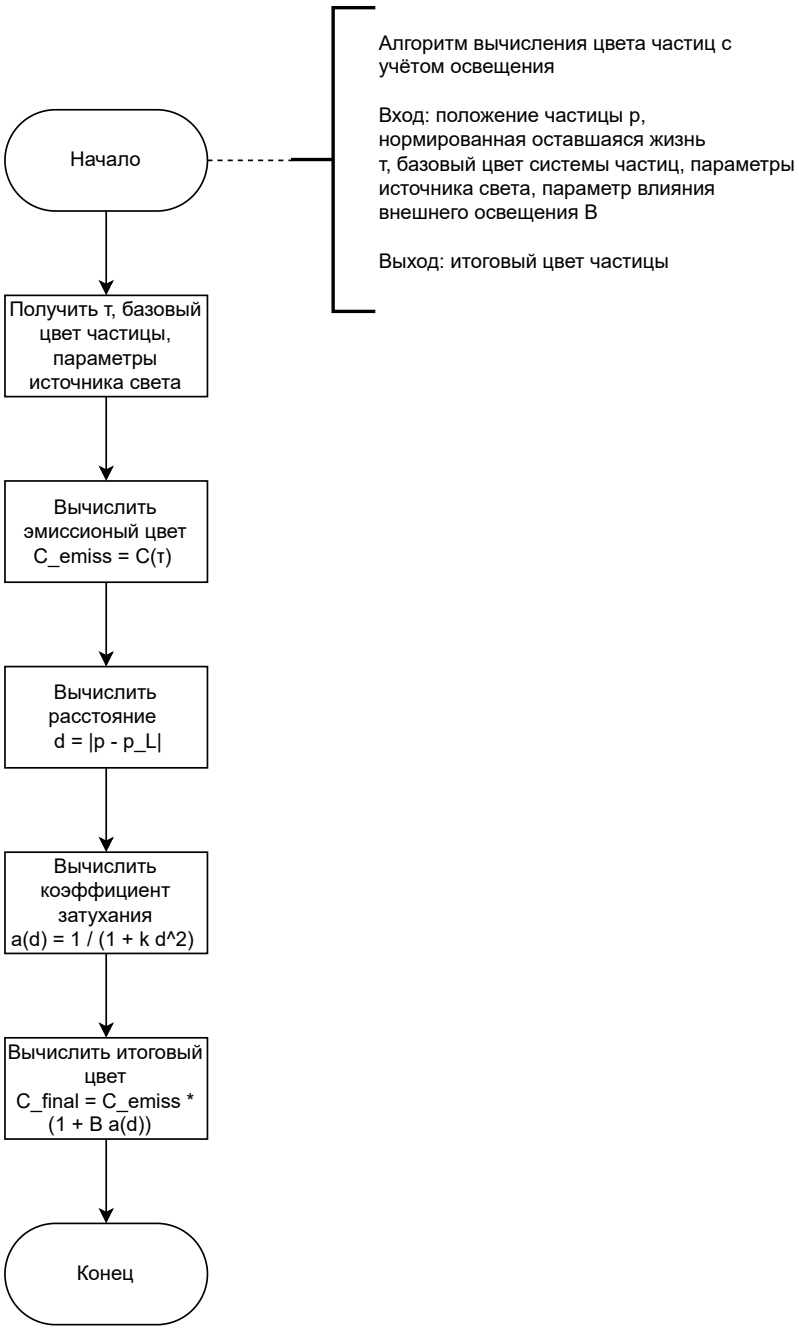


Рисунок 2.4 — Схема алгоритма вычисления цвета частицы с учётом освещения

2.7 Алгоритм визуализации сцены

Алгоритм визуализации сцены описывает преобразование трёхмерных координат частиц в двумерное изображение с учётом перспективной проекции и удаления невидимых поверхностей.

Схема алгоритма визуализации сцены с использованием z-буфера приведена на рисунке 2.5.

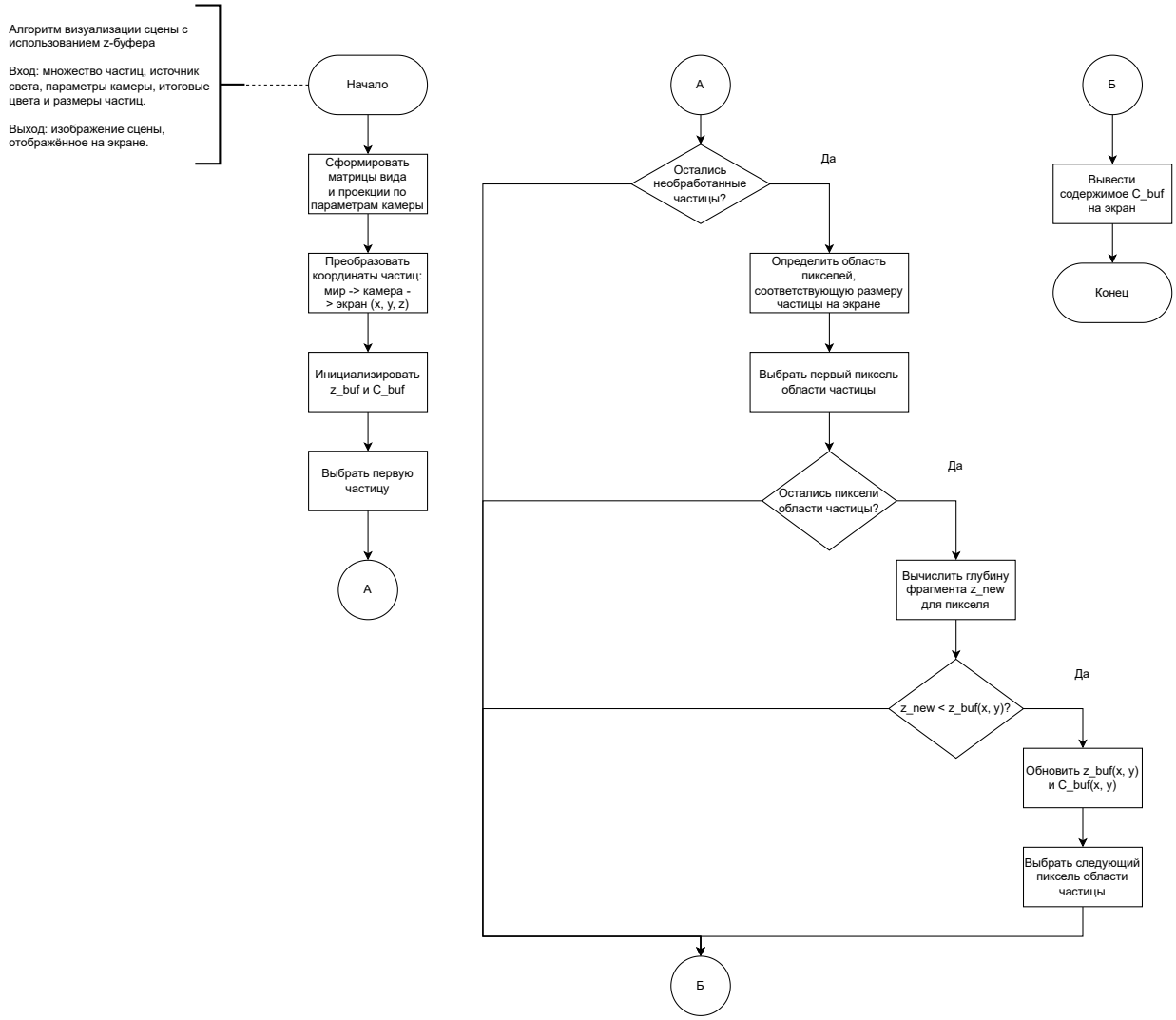


Рисунок 2.5 — Схема алгоритма визуализации сцены с использованием z-буфера

2.8 Вывод

В конструкторской части сформулированы требования к программному обеспечению для визуализации трёхмерного огня методом систем частиц, разработана общая архитектура программного комплекса и формальная модель данных, описывающая основные объекты сцены.

Для ключевых алгоритмов работы программы (основной цикл, обновление систем частиц, вычисление цветов с учётом освещения, проекция и удаление невидимых поверхностей) приведены формальные описания.

3 Технологическая часть

В технологическом разделе рассматриваются инструменты реализации и разработана архитектура программного обеспечения, декомпозиция программы, ключевые алгоритмы и пользовательский интерфейс.

3.1 Выбор языка программирования и средств разработки

Для реализации программного обеспечения был выбран язык программирования C++ и кроссплатформенный фреймворк Qt. Данный выбор обусловлен следующими соображениями:

1) язык C++ был изучен при прохождении курса "Объектно-ориентированное программирование";

2) объектно-ориентированный подход позволяет естественным образом представить сущности системы (частицы, системы частиц, камеру, источники света) в виде классов с инкапсуляцией данных и поведения;

3) библиотека Qt предоставляет [7]:

- средства для создания графического интерфейса (Qt Widgets);
- кроссплатформенность (Windows, Linux, macOS);
- встроенную поддержку 2D графики (QPainter, QImage);
- механизмы сигналов и слотов для обработки событий.

В качестве среды разработки использовался Qt Creator 8.0.1, система сборки – CMake 3.25. Проект компилируется с использованием стандарта C++17.

3.2 Организация модулей программы

Программа состоит из следующих модулей:

1) математические модули:
   - `vector3.h/cpp` – трёхмерные векторы и операции;
   - `matrix4x4.h/cpp` – матрицы 4×4 и преобразования.
2) модули моделирования:
   - `particle.h/cpp` – класс отдельной частицы;
   - `particlesystem.h/cpp` – система частиц;
   - `scene.h/cpp` – управление сценой;
   - `camera.h/cpp` – виртуальная камера;
   - `light.h/cpp` – источник освещения.
3) модули визуализации:
   - `firecanvas.h/cpp` – виджет рендеринга с реализацией z-буфера.
4) модули интерфейса:
   - `mainwindow.h/cpp` – главное окно приложения;
   - `main.cpp` – точка входа.

На рисунке 3.1 представлена диаграмма компонентов программы.

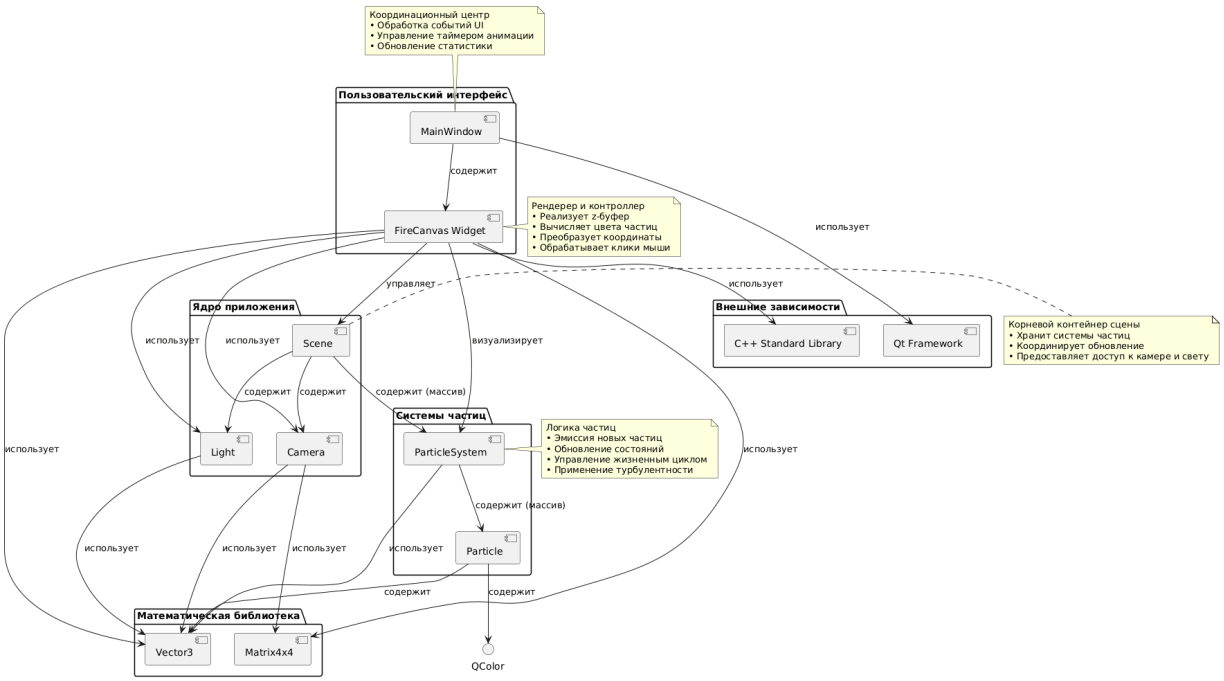


Рисунок 3.1 — Диаграмма компонентов системы визуализации огня

3.3 Структура классов программного обеспечения

На рисунке 3.2 представлена диаграмма классов основных сущностей системы.

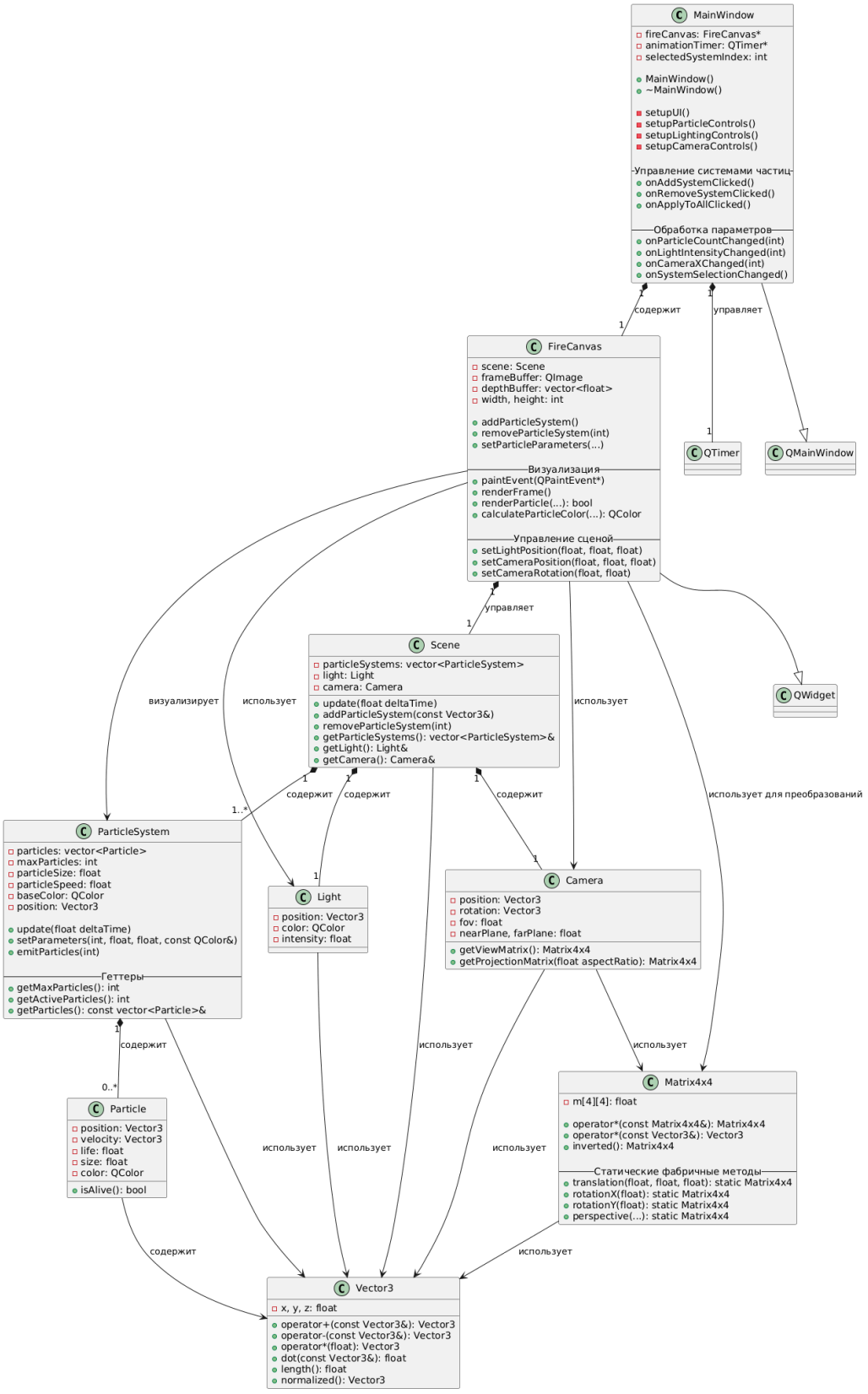


Рисунок 3.2 — Диаграмма классов системы визуализации огня

3.4 Реализация ключевых алгоритмов

3.4.1 Алгоритм обновления системы частиц

Метод update класса ParticleSystem реализует обновление состояния системы частиц за временной шаг. В листинге 3.1 представлена реализация данного метода.

```
void ParticleSystem::update(float deltaTime) {
  timeSinceLastEmission += deltaTime;
  int particlesToEmit = static_cast<int>(emissionRate * deltaTime);
  if (particlesToEmit > 0 &&
  particles.size() < static_cast<size_t>(maxParticles)) {
    emitParticles(particlesToEmit);
  }

  for (auto &particle : particles) {
    if (!particle.isAlive()) continue;

    particle.position = particle.position +
    particle.velocity * deltaTime * particleSpeed;

    const float decayRate = 0.4f;
    particle.life -= decayRate * deltaTime;

    if (!particle.isAlive()) continue;

    particle.velocity.x += (std::rand() % 100 - 50) / 500.0f *
       deltaTime;
    particle.velocity.z += (std::rand() % 100 - 50) / 500.0f *
       deltaTime;

    updateParticleVisuals(particle);
  }

  if (std::rand() % 1000 == 0) {
    particles.erase(std::remove_if(particles.begin(), particles.end()
       ,
    [](const Particle &p) { return !p.isAlive(); }),
    particles.end());
  }
}
```

Листинг 3.1 — Реализация метода обновления системы частиц

3.4.2 Алгоритм вычисления цвета частицы с учётом освещения

Метод `calculateParticleColor` класса `FireCanvas` реализует вычисление конечного цвета частицы на основе принятой модели освещения. В листинге 3.2 представлена реализация данного метода.

```
QColor FireCanvas::calculateParticleColor(const Particle &particle) {
  Light &light = scene.getLight();

  QColor baseColor = particle.color;
  float life = std::max(0.0f, std::min(1.0f, particle.life));
  baseColor.setAlphaF(life);

  Vector3 toLight = light.position - particle.position;
  float distanceToLight = toLight.length();

  float attenuation = 1.0f / (1.0f + 0.01f *
  distanceToLight * distanceToLight);

  float emissionFactor = 1.0f;
  float externalFactor = light.intensity * attenuation;
  float totalFactor = emissionFactor + externalFactor;

  float r = baseColor.redF() * totalFactor;
  float g = baseColor.greenF() * totalFactor;
  float b = baseColor.blueF() * totalFactor;
  float a = baseColor.alphaF();

  r = std::min(1.0f, r);
  g = std::min(1.0f, g);
  b = std::min(1.0f, b);

  QColor result;
  result.setRgbF(r, g, b, a);
  return result;
}
```

Листинг 3.2 — Вычисление цвета частицы с учётом освещения

3.4.3 Алгоритм z-буфера с аддитивным смешиванием

Метод `renderParticle` класса `FireCanvas` реализует рендеринг отдельной частицы с использованием алгоритма z-буфера. В листинге 3.3 представлена реализация данного метода.

```
bool FireCanvas::renderParticle(const Particle &particle,
const Matrix4x4 &viewProjection) {
  Vector3 screenPos = viewProjection * particle.position;

  if (screenPos.z < 0 || screenPos.z > 1 ||
  screenPos.x < -1 || screenPos.x > 1 ||
  screenPos.y < -1 || screenPos.y > 1) {
    return false;
  }

  int x = (screenPos.x + 1) * width / 2;
  int y = (1 - screenPos.y) * height / 2;

  int index = y * width + x;
  if (screenPos.z < depthBuffer[index]) {
    depthBuffer[index] = screenPos.z;

    drawBillboard(x, y, screenPos.z,
    static_cast<int>(particle.size * 100),
    calculateParticleColor(particle));
    return true;
  }
  return false;
}
```

Листинг 3.3 — Рендеринг частицы с использованием z-буфера

3.5 Интерфейс пользователя

Интерфейс программы разделён на логические блоки управления, организованные во вкладках. На рисунках 3.3–3.7 представлены основные элементы интерфейса.

Область визуализации (левый блок) отображает 3D-сцену с системами частиц в реальном времени. Панель управления (правый блок) содержит четыре вкладки: «Частицы», «Свет», «Камера» и «Статистика».

Пользователь может добавлять новые системы частиц по клику в области визуализации. Все изменения параметров отражаются на визуализации, обеспечивая работу в реальном времени. Интерфейс использует тёмную цветовую схему для снижения нагрузки на глаза при длительной работе.

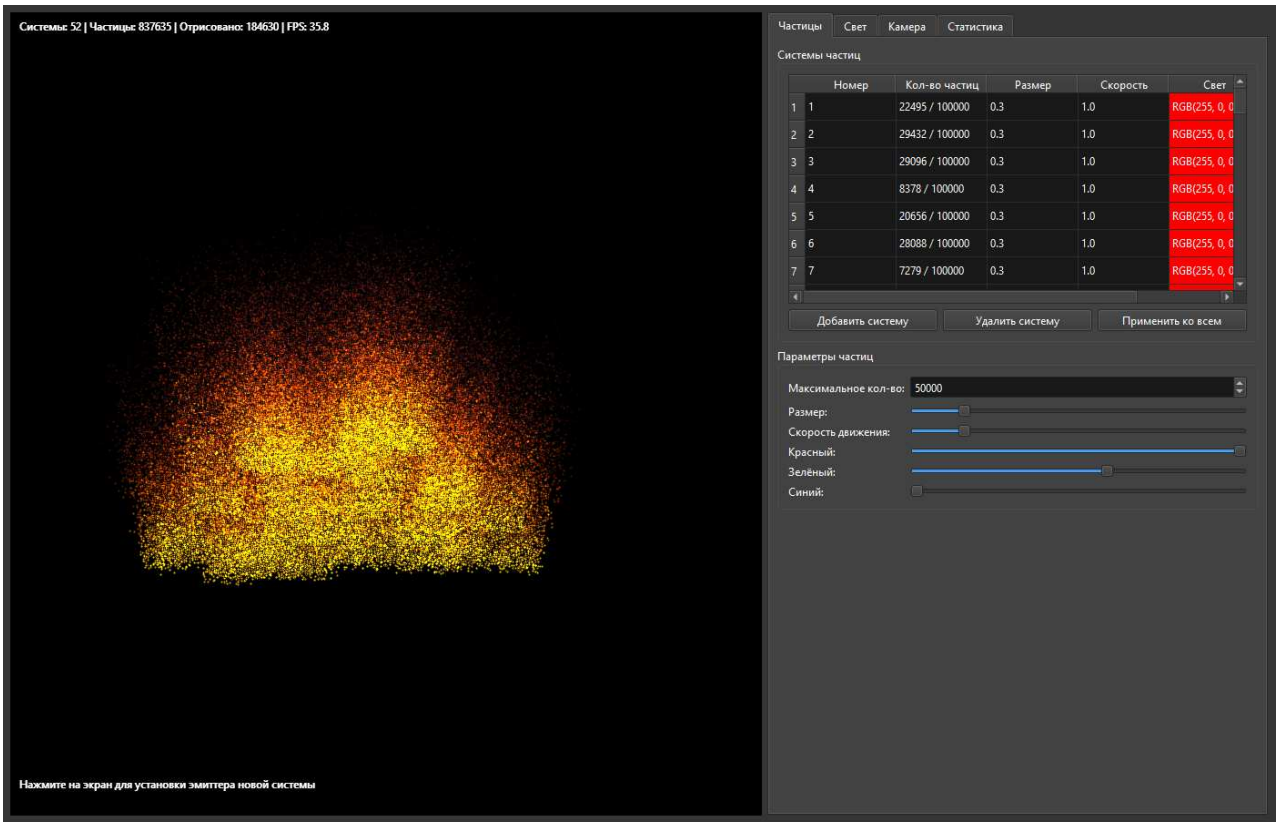


Рисунок 3.3 — Главное окно программы с областью визуализации и панелью управления

Вкладка "Частицы"содержит таблицу систем частиц с текущими параметрами, элементы управления параметрами выбранной системы, кнопки добавления и удаления систем, а также ползунки для настройки цвета (RGB). Интерфейс позволяет управлять несколькими независимыми системами частиц, каждая из которых представляет отдельный источник пламени.

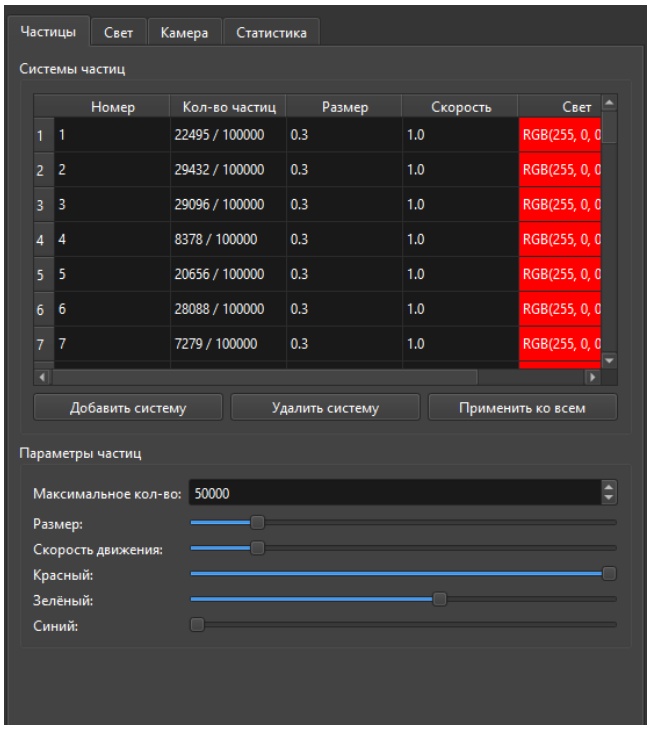


Рисунок 3.4 — Вкладка управления частицами с таблицей систем и элементами настройки

Вкладка "Свет"предоставляет элементы для настройки позиции источника света (координаты X, Y, Z), регулятор интенсивности освещения и отображение текущих координат. Пользователь может интерактивно изменять параметры внешнего освещения и наблюдать их влияние на визуализацию пламени.

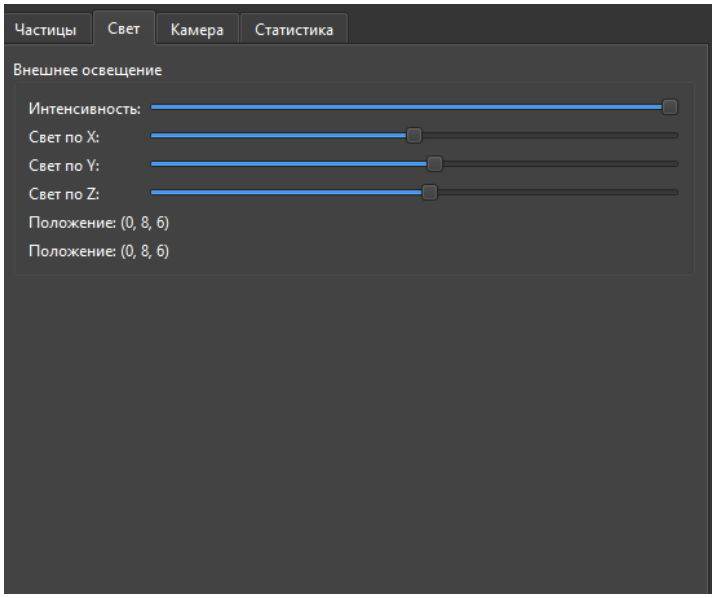


Рисунок 3.5 — Вкладка управления освещением с настройками источника света

Вкладка "Камера"содержит элементы управления позицией камеры, регулировки углов поворота и настройки поля зрения (FOV). Это позволяет пользователю свободно перемещаться по сцене и выбирать оптимальный ракурс для наблюдения

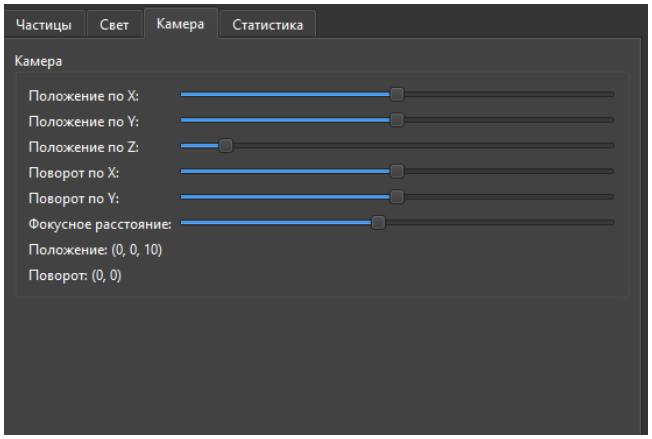


Рисунок 3.6 — Вкладка управления камерой с настройками позиции и ориентации

Вкладка "Статистика"отображает текущий FPS, общее количество частиц и количество активных систем. Эта информация позволяет отслеживать производительность программы при различных настройках и количестве частиц.

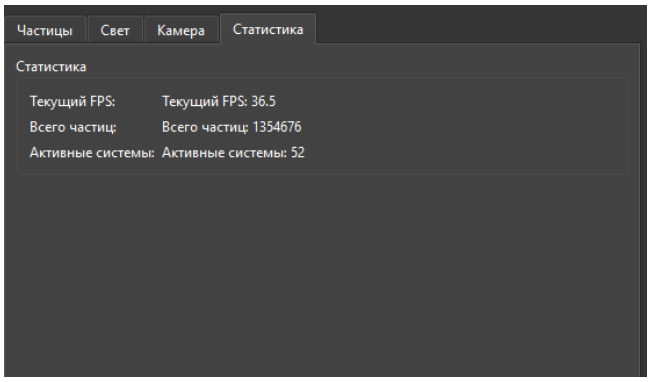


Рисунок 3.7 — Вкладка статистики с информацией о производительности системы

3.6 Результаты работы программного обеспечения

Программное обеспечение позволяет визуализировать трёхмерный огонь с различными параметрами систем частиц, освещения и камеры. На рисунках 3.8–3.13 представлены примеры работы программы.

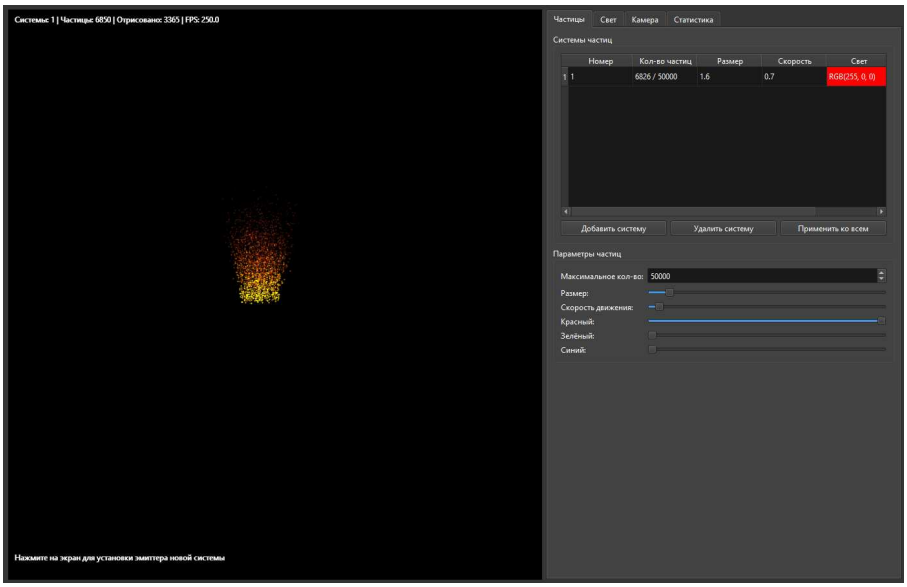


Рисунок 3.8 — Визуализация одной системы частиц

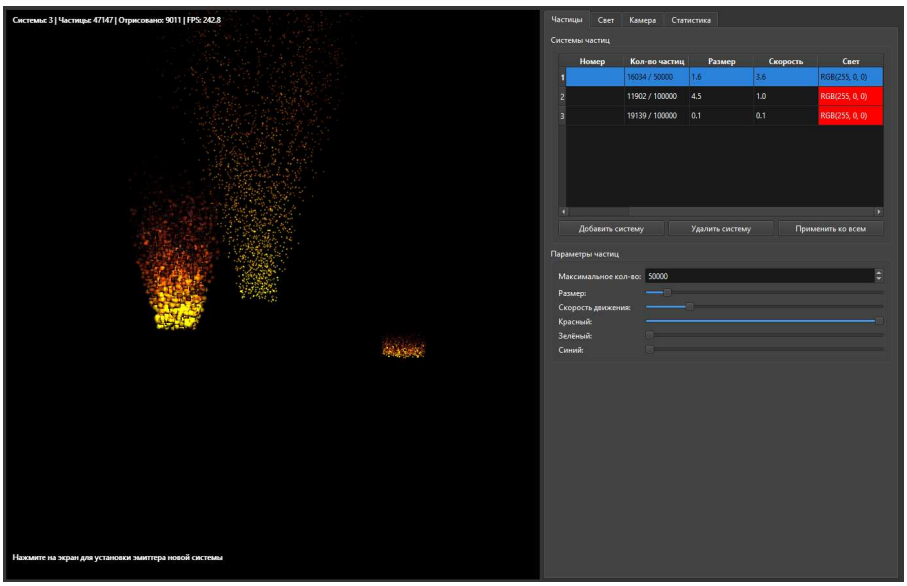


Рисунок 3.9 — Несколько систем частиц с разными параметрами

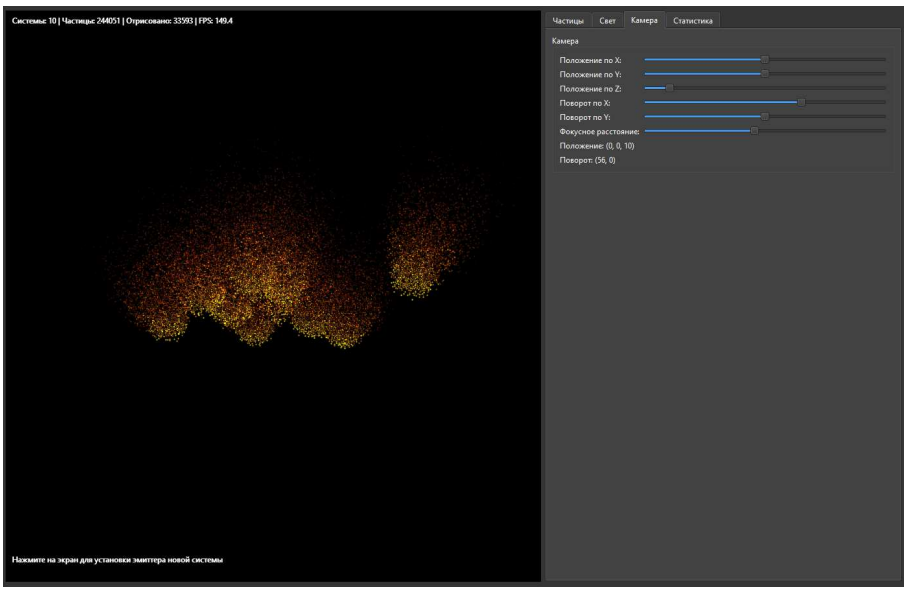


Рисунок 3.10 — Изменение ракурса камеры

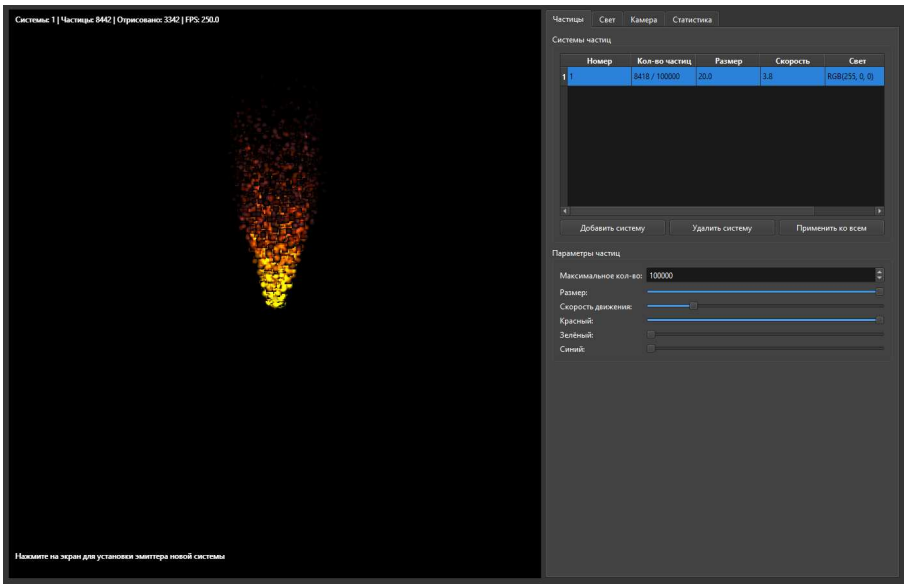


Рисунок 3.11 — Увеличенный размер частиц

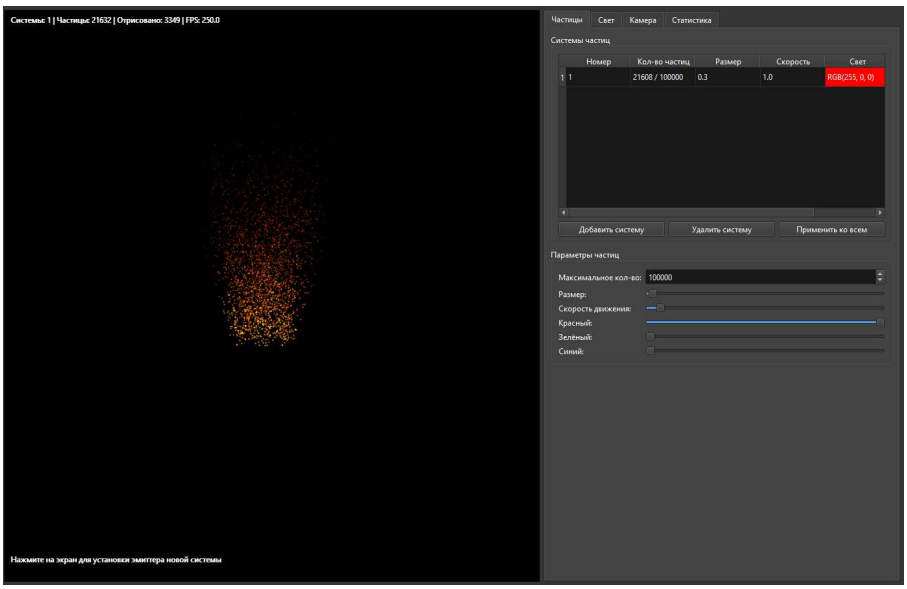


Рисунок 3.12 — Визуализация без внешнего освещения

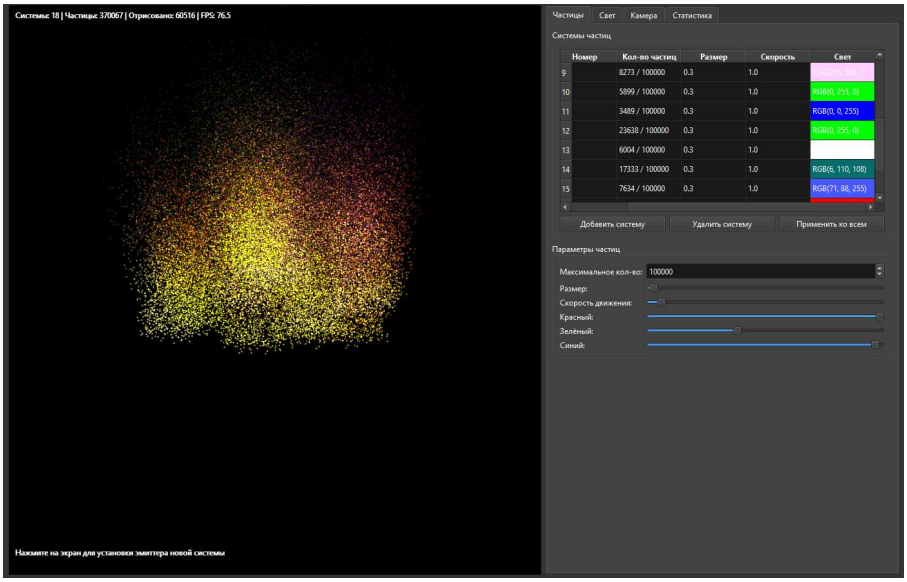


Рисунок 3.13 — Визуализация множества систем различного цвета

3.7 Вывод

В технологической части разработана архитектура программного комплекса для визуализации трёхмерного огня. Выбран язык программирования C++ и фреймворк Qt для реализации. Разработана объектно-ориентированная архитектура с разделением ответственности между классами. Реализованы ключевые алгоритмы.

Создан графический интерфейс с разделением на логические блоки управления. Обеспечена возможность интерактивного управления сценой в реальном времени со статистикой производительности.

4 Исследовательская часть

В исследовательской части проводится изучение производительности системы визуализации трёхмерного огня.

4.1 Методика исследования производительности

Целью исследования является анализ влияния количества частиц на производительность системы визуализации. Исследование включает изучение зависимости частоты кадров от общего количества частиц, влияние внешнего освещения, а также наличие алгоритма z-буфера.

Параметры исследования:

1) количество частиц в сцене: от 10 000 до 2 000 000;
2) наличие внешнего освещения;
3) использование алгоритма z-буфера.

Изменение производительности вычисляется по формуле:

$$\Delta P = \left(1 - \frac{\text{FPS}_{\text{результат1}}}{\text{FPS}_{\text{результат2}}}\right) \times 100\%,$$

Измерения проводились на системе с процессором Intel Core i9-14900K (24 ядра, 8 логических ядер) [9], видеокартой NVIDIA GeForce RTX 4090 и 64 ГБ оперативной памяти. Все тесты выполнялись при разрешении 800×600 пикселей.

4.2 Анализ производительности системы

4.2.1 Зависимость частоты кадров от количества частиц

На рисунке 4.1 и в таблице 4.1 представлена зависимость частоты кадров от количества частиц.

Таблица 4.1 — Зависимость частоты кадров от количества частиц

Количество частиц	**FPS (без освещения)**	**FPS (с освещением)**	**Снижение, %**
10 000	240	220	8.3
50 000	180	155	13.9
100 000	120	100	16.7
500 000	45	35	22.2
1 000 000	22	16	27.3
2 000 000	11	8	27.3

С увеличением количества частиц частота кадров закономерно снижается. При использовании внешнего освещения наблюдается дополнительное снижение производительности. Увеличение количества частиц с 10 000 до 2 000 000 приводит к снижению частоты кадров более чем в 20 раз. Включение внешнего освещения дополнительно снижает производительность на 8–27%.

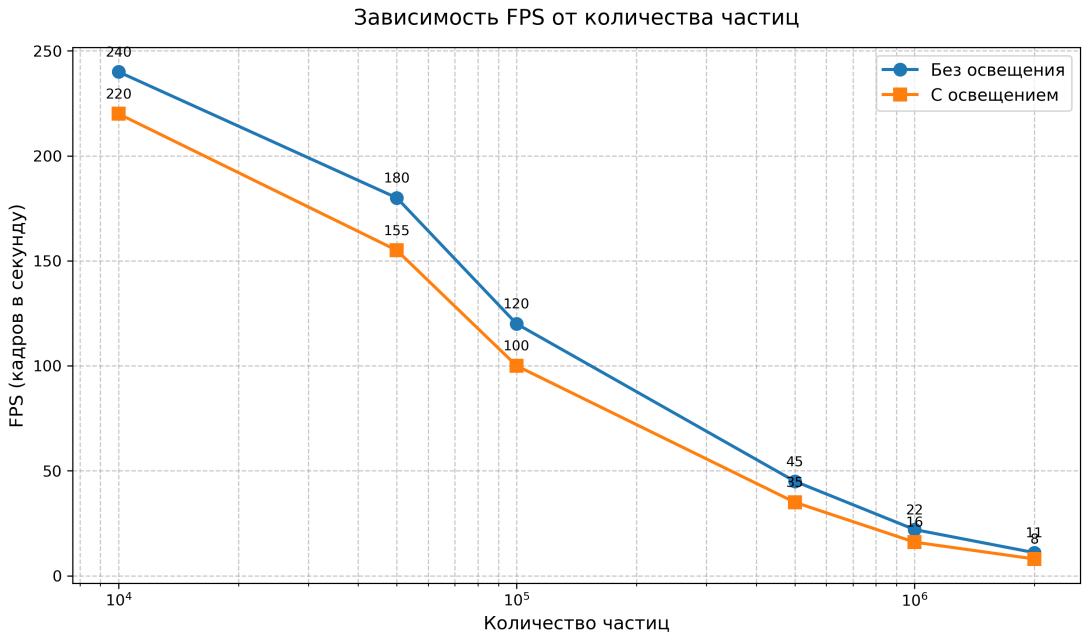


Рисунок 4.1 — График зависимости частоты кадров от количества частиц

4.2.2 Зависимость от наличия z-буфера

Для оценки влияния алгоритма z-буфера проведено сравнение производительности при его использовании и без него. Результаты представлены в таблице 4.2 и на рисунке 4.2.

Таблица 4.2 — Сравнение производительности с использованием z-буфера и без него (100 000 частиц)

Режим рендеринга	**FPS (без освещения)**	**FPS (с освещением)**	**Затраты, %**
С использованием z-буфера	120	100	–
Без использования z-буфера	160	135	25.0–35.0

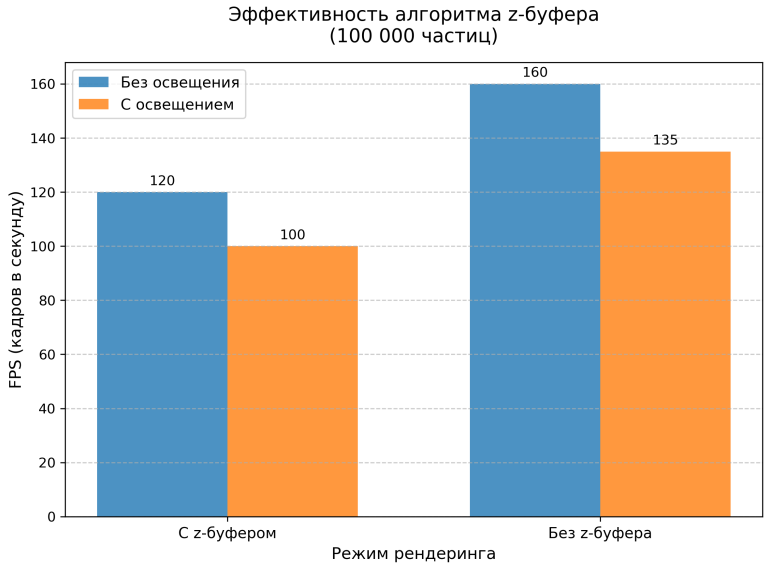


Рисунок 4.2 — График сравнения производительности с использованием z-буфера и без него

Аалгоритм z-буфера обеспечивает корректное отображение частиц по глубине, но вносит дополнительные вычислительные затраты в размере 25–35% по сравнению с рендерингом без проверки глубины. При отключении z-буфера производительность возрастает, однако нарушается визуализация: частицы, находящиеся дальше от камеры, могут отображаться поверх более близких.

4.3 Вывод

В рамках исследовательской части проведено изучение производительности системы визуализации трёхмерного огня: проанализировано влияние количества частиц на частоту обновления кадров, исследована эффективность алгоритма z-буфера и оценено влияние внешнего освещения на производительность.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы была достигнута поставленная цель – разработано программное обеспечение для визуализации трёхмерного огня методом систем частиц с учётом внешнего освещения и проведено исследование влияния ключевых параметров системы на производительность. Все поставленные задачи успешно выполнены.

1) Проведён сравнительный анализ существующих методов визуализации и моделей освещения. На его основе выбрана и обоснована эмиссионная модель освещения, которая сочетает учёт самосвечения частиц с влиянием внешнего источника света через функцию затухания. Для решения задачи видимости выбран алгоритм z-буфера с поддержкой аддитивного смешивания, что обеспечивает корректное отображение полупрозрачных частиц и высокую производительность при большом количестве примитивов.

2) Разработана структура программного обеспечения с чётким разделением ответственности между компонентами. Спроектированы основные классы для представления частицы, системы частиц, камеры, источника света и сцены, что обеспечило модульность и расширяемость системы.

3) Реализован программный комплекс на C++ с использованием фреймворка Qt. Программа предоставляет интерактивный интерфейс, позволяющий пользователю в реальном времени создавать, настраивать и управлять несколькими независимыми системами частиц, а также изменять параметры внешнего освещения и виртуальной камеры.

4) Проведено исследование производительности системы. Установлена закономерная зависимость частоты кадров от общего количества частиц в сцене. Проанализировано влияние внешнего освещения и алгоритма z-буфера на производительность. Результаты показывают, что выбранные алгоритмы обеспечивают приемлемую частоту кадров при визуализации сотен тысяч частиц, демонстрируя баланс между реалистичностью и быстродействием.

Таким образом, разработанное программное обеспечение подтверждает работоспособность выбранного подхода и решает задачу интерактивной визуализации трёхмерного огня с учётом внешнего освещения. Результаты работы могут быть использованы в дальнейшем для оптимизации алгоритмов, внедрения более сложных моделей освещения или адаптации системы для моделирования других схожих явлений (дыма, взрывов, тумана).

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Reeves, W. T. Particle systems — a technique for modeling a class of fuzzy objects / W. T. Reeves // Computer Graphics. — 1983. — Vol. 17, № 3. — P. 359–376.

2. Дёмин, А. Ю. Основы компьютерной графики : учеб. пособие / А. Ю. Дёмин. – Томск : Изд-во Томского политехнического университета, 2011. — 191 с.

3. Роджерс, Д. Ф. Математические основы машинной графики : пер. с англ. / Д. Ф. Роджерс, Дж. А. Адамс. — 2-е изд., перераб. и доп. — М. : Мир, 2001. — 604 с.

4. Фоли, Дж. Основы интерактивной машинной графики : в 2 кн. / Дж. Фоли, А. ван Дам ; пер. с англ. — М. : Мир, 1985. — 2 кн.

5. Kaufman, A. E. Volume Visualization / A. E. Kaufman. — Los Alamitos : IEEE Computer Society Press, 1991. — 479 p.

6. Eck, D. J. Introduction to Computer Graphics [Электронный ресурс] / D. J. Eck. — Version 1.4. — Hobart and William Smith Colleges, 2015–2023. — Режим доступа: `http://math.hws.edu/graphicsbook/` (дата обращения: 23.11.2025).

7. Qt Documentation [Электронный ресурс]. — Режим доступа: `https://doc.qt.io/` (дата обращения: 03.11.2025).

8. ISO/IEC 14882:2017. Information technology — Programming languages – C++ [Электронный ресурс]. — 5th ed. — Geneva : ISO, 2017-12. — 1605 p. — Режим доступа: `https://www.iso.org/standard/68564.html` (дата обращения: 04.11.2025).

9. Intel® 64 and IA-32 Architectures Software Developer's Manual: Vol. 3B. System Programming Guide, Part 2 [Электронный ресурс]. — October 2025. — Режим доступа: `https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html` (дата обращения: 06.11.2025).

ПРИЛОЖЕНИЕ А

Презентация к курсовой работе.